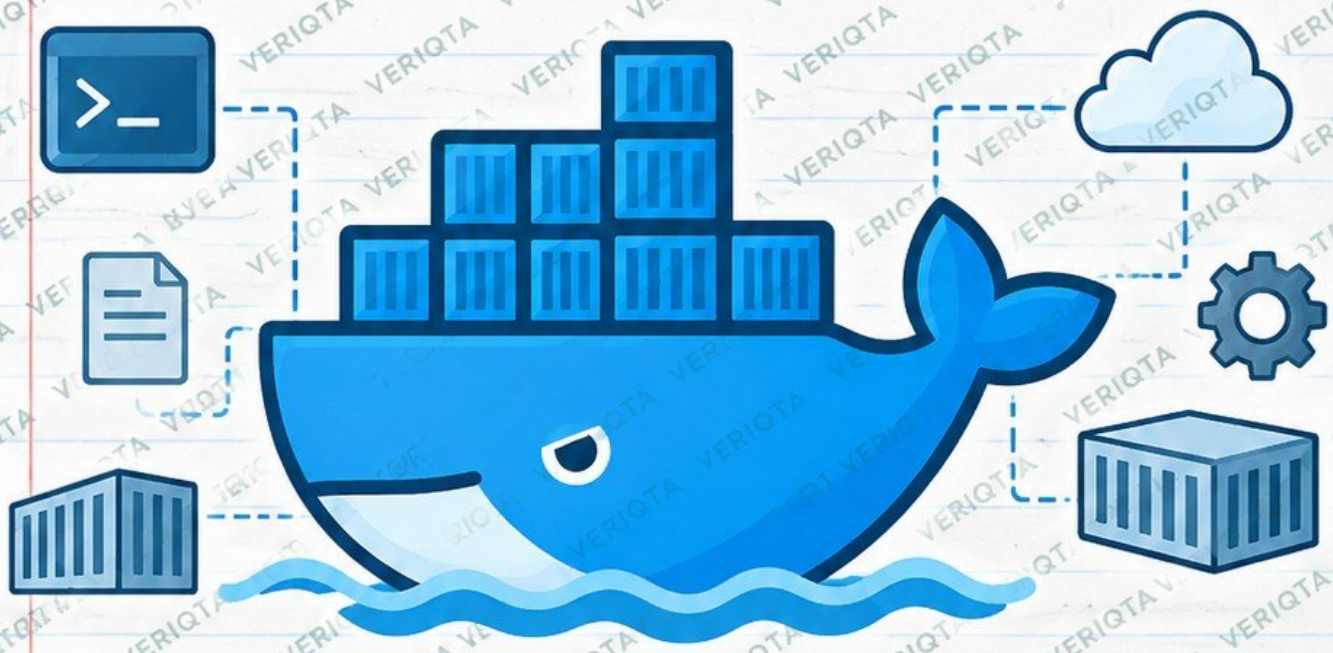
VERIQTA

DOCKER

COMMANDS CHEAT SHEETS



ESSENTIAL COMMANDS | PRACTICAL EXAMPLES | TROUBLESHOOTING TIPS | REAL-WORLD USAGE

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

DOCKER

INSTALLATION AND ENVIRONMENT CHECKS

1. CHECK DOCKER CLIENT VERSION

Shows the installed Docker CLI version.

```
$ docker --version
Docker version 26.1.2, build 211e74b
```

WHEN TO USE IT

- To confirm Docker is installed correctly.
- To quickly check the client version.

2 CHECK DETAILED VERSION INFORMATION

Shows both client and server (engine) versions.

```
$ docker version
Client: Docker Engine - Community
 Version:           26.1.2
Server: Docker Engine - Community
 Version:           26.1.2
```

WHEN TO USE IT

- To confirm the Docker client and daemon versions.
- Useful for troubleshooting version mismatches.

3 VIEW DOCKER SYSTEM INFORMATION

Shows detailed information about the Docker engine, containers, storage, etc.

```
$ docker info
Server:
 Containers: 2
 Images: 5
 Storage Driver: overlay2
 Operating System: Ubunta 24.04 LTS
 CPUs: 4
 Total Memory: 7.6GiB
```

WHEN TO USE IT

- To check the health and configuration of Docker.
- Useful for troubleshooting and verifying the environment.

4 SHOW AVAILABLE COMMANDS

Displays a list of Docker commands and options.

```
$ docker --help
Usage:  docker [OPTIONS] COMMAND
A self-sufficient runtime for containers.
```

WHEN TO USE IT

- To explore available Docker commands.
- Useful when you forget the syntax or want to learn more.

5 SHOW SYSTEM-WIDE INFORMATION

Provides low-level system information.

```
$ docker system info
System:
  Containers: 2
  Images: 5
  Server Version: 26.1.2
  Storage Driver: overlay2
```

WHEN TO USE IT

- To get detailed system-wide information.
- Useful for advanced troubleshooting.

DOCKER CLI vs DOCKER ENGINE

Docker CLI		Docker Engine
• Command-line tool (docker) • You run commands here • Sends requests to the Docker Engine • Installed on your machine	Sends commands to →	• Runs in the background • Manages containers, images, networks, volumes • Called the Docker daemon • Performs the actual work

TROUBLESHOOTING DOCKER DAEMON CONNECTIVITY

COMMON ERROR

```
$ docker ps
Cannot connect to the Docker daemon
at unix:///var/run/docker.sock.
Is the docker daemon running?
```

POSSIBLE CAUSES

- Docker daemon is not running.
- Docker service not started.
- Permission issues.
- Docker not installed correctly.
- Incorrect Docker socket path.

HOW TO FIX

1. Start Docker service — `$ sudo systemctl start docker`
2. Check service status — `$ sudo systemctl status docker`
3. Verify connectivity — `$ docker ps`

DOCKER

IMAGES, SEARCH AND INSPECTION



1 SEARCH FOR IMAGES

Search Docker Hub for images.

```
$ docker search nginx
```

NAME	DESCRIPTION	STARS	OFFICIAL
nginx	Official build of NGINX	20345	[OK]
nginxinc/nginx-unprivileged	Unprivileged NGINX	102	
bitnami/nginx	NGINX packaged by Bitnami	194	

WHEN TO USE IT

- To find official or community images.
- To see available tags and image options.

2 PULL AN IMAGE

Download an image from a registry (Docker Hub).

```
$ docker pull nginx:alpine
alpine: Pulling from library/nginx
Digest: sha256:3a2b...
Status: Downloaded newer image for nginx:alpine
```

WHEN TO USE IT

- To download an image to your local machine.
- Specify a tag to get a specific version.

3 LIST LOCAL IMAGES

Show all downloaded images.

```
$ docker image ls
```

REPOSITORY	TAG	IMAGE ID	SIZE
nginx	alpine	a1b2c3d4e5f6	55.3MB
ubuntu	24.04	4f5e6d7c8b9a	78.1MB

WHEN TO USE IT

- To see images available on your machine.
- Useful before running or removing images.

4 INSPECT AN IMAGE

Show detailed configuration of an image.

```
$ docker image inspect nginx:alpine
```

```
[{
    "Id": "sha256:a1b2c3d4e5f6...",
    "RepoTags": ["nginx:alpine"],
    "Size": 55334821,
    "Architecture": "amd64",
    "Os": "linux"
}]
```

WHEN TO USE IT

- To view metadata, environment variables, exposed ports, layers, and more.

5 VIEW IMAGE HISTORY

Show the layers used to build an image.

```
$ docker image history nginx:alpine
```

IMAGE	CREATED	CREATED BY	SIZE
a1b2c3d4e5f6	2 weeks ago	/bin/sh -c #(nop)...	0B
b2c3d4e5f6g7	2 weeks ago	/bin/sh -c apk add...	8.3MB
c3d4e5f6g7h8	2 weeks ago	/bin/sh -c #(nop)...	0B

WHEN TO USE IT

- To see the layers and commands used to build the image.
- Helpful for understanding image size and content.

6 REMOVE AN IMAGE

Delete an image from your local machine.

```
$ docker image rm nginx:alpine
Untagged: nginx:alpine
Deleted: sha256:a1b2c3d4e5f6...
```

WHEN TO USE IT

- To free up disk space.
- Use with caution, especially if containers are using it.

7 IMAGE TAGS, REGISTRIES AND IMAGE IDs

IMAGE TAGS

- Identify a specific version
- Format: name:tag
- Example: nginx:alpine, nginx:1.25, nginx:latest
- If no tag is specified, latest is used by default.

REGISTRIES

- Docker Hub: default public registry
- Private registries: used by companies
- Format: registry/name:tag
- Example: nginx:alpine
- Example: myregistry.com/app:1.0

IMAGE IDs

- Unique identifier for each image
- Format: long SHA256 hash
- Example: a1b2c3d4e5f6...
- You can use part of the ID
- Useful when tags are missing

JUNIOR ENGINEER TIP

Always use specific tags (e.g., **alpine** or a version number) instead of **latest** in production to ensure consistency and avoid unexpected changes.

DOCKER

RUNNING YOUR FIRST CONTAINER

1 RUN A TEST CONTAINER

Runs a small test container to verify Docker is working.

```
$ docker run hello-world
```

```
Hello from Docker!
This message shows that your installation
appears to be working correctly.
```

WHEN TO USE IT

- To confirm Docker is installed and working.
- Useful for first-time setup or troubleshooting.

2 RUN A CONTAINER IN DETACHED MODE

Runs an Nginx web server in the background.

```
$ docker run -d --name web nginx:alpine
a1b2c3d4e5f67890abcd1234567890efgh
```



WHEN TO USE IT

- To run services like web servers, databases, etc.
- Container keeps running in the background.

3 RUN AN INTERACTIVE CONTAINER

Starts an Ubuntu container with an interactive shell.

```
$ docker run -it ubuntu:24.04 bash
root@a1b2c3d4e5f6:/#
```

WHEN TO USE IT

- To explore a container interactively.
- Useful for learning, testing and debugging.

4 RUN A CONTAINER AND REMOVE IT

Runs a command and automatically removes the container when it exits.

```
$ docker run --rm alpine:3.20 echo hello
hello
```

WHEN TO USE IT

- For one-time tasks or quick tests.
- Keeps your system clean by removing the container after it exits.

5 MAP CONTAINER PORT TO HOST

Runs an Nginx container and maps port 80 to port 8080 on your machine.

```
$ docker run -p 8080:80 nginx:alpine
a1b2c3d4e5f6e7f8g9h0...
```

WHEN TO USE IT

- To access the container service from your browser.
- Visit: http://localhost:8080

6 DETACHED vs INTERACTIVE CONTAINERS

Understand the difference and when to use each.

DETACHED MODE (-d)

- Runs in the background.
- You get your terminal back.
- Useful for long-running services.

Use case: Web servers, databases, background services.

INTERACTIVE MODE (-it)

- Opens an interactive shell.
- You can run commands inside the container.
- Useful for exploration and debugging.

Use case: Learning, testing, manual troubleshooting.

JUNIOR ENGINEER TIPS

- Use `--name` to give your container a meaningful name.
- Use `--rm` for temporary containers.
- Use `-p` to expose container ports to your machine.

COMMON MISTAKES

- Forgetting `-d` and wondering why the terminal is stuck.
- Using the wrong port mapping.
- Trying to access a container without `-p`.
- Not using `--rm` for temporary containers.

DOCKER

CONTAINER LIFECYCLE MANAGEMENT

1 LIST RUNNING CONTAINERS

Shows only containers that are currently running.

```
$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	STATUS	NAMES
a1b2c3d4e5f6	nginx	"nginx -g.."	Up 5 minutes	web
b2c3d4e5f6g7	ubuntu	"bash"	Up 2 hours	dev

WHEN TO USE IT

- To see all active containers.
- Useful for checking which services are running.

2 LIST ALL CONTAINERS

Shows both running and stopped containers.

```
$ docker ps -a
```

CONTAINER ID	IMAGE	STATUS	NAMES
a1b2c3d4e5f6	nginx	Up 5 minutes	web
b2c3d4e5f6g7	ubuntu	Up 2 hours	dev
c3d4e5f6g7h8	redis	Exited (0) 10 minutes ago	cache

WHEN TO USE IT

- To see all containers, including stopped ones.
- Useful before starting, removing or troubleshooting.

3 START A CONTAINER

Starts a stopped container.

```
$ docker start CONTAINER
$ docker start web
```

WHEN TO USE IT

- To start a container that is stopped.
- Useful when you want to reuse an existing container.

4 STOP A CONTAINER

Stops a running container gracefully.

```
$ docker stop CONTAINER
$ docker stop web
```

WHEN TO USE IT

- To stop a container normally.
- Gives the container time to shut down properly.

5 RESTART A CONTAINER

Stops and starts a container.

```
$ docker restart CONTAINER
$ docker restart web
```

WHEN TO USE IT

- To quickly apply changes or recover a service.
- Useful after configuration updates.

6 KILL A CONTAINER

Stops a container immediately (force stop).

```
$ docker kill CONTAINER
$ docker kill web
```

WHEN TO USE IT

- To forcefully stop an unresponsive container.
- Use only when a normal stop does not work.

7 REMOVE A CONTAINER

Deletes a stopped container.

```
$ docker rm CONTAINER
$ docker rm web
```

WHEN TO USE IT

- To remove containers you no longer need.
- Container must be stopped before removal.

STOPPED vs RUNNING CONTAINERS

RUNNING CONTAINER

- Actively executing processes
- Appears in `docker ps`
- Uses system resources
- Application is accessible
- STATUS shows "Up ..."



STOPPED CONTAINER

- Not executing processes
- Does not appear in `docker ps`
- No system resources used
- Application is not accessible
- STATUS shows "Exited .."



JUNIOR ENGINEER TIP

- Use `docker ps` to check running containers daily.
- Use `docker ps -a` to find containers you can remove.
- Remove unused containers to free up disk space.

COMMON MISTAKES

- Trying to remove a running container (use `docker stop` first).
- Using `docker kill` when a normal stop is enough.
- Forgetting that stopped containers still exist and use disk space.

DOCKER

CONTAINER INSPECTION AND TROUBLESHOOTING

1 VIEW CONTAINER LOGS

Shows the output and error logs from a container.

```
$ docker logs CONTAINER
```

```
$ docker logs web
/docker-entrypoint.sh: Starting Nginx...
/docker-entrypoint.sh: Configuration
complete; ready for start up
```

WHEN TO USE IT

- To see application logs and errors.
- Useful for troubleshooting issues.

2 FOLLOW LOGS IN REAL TIME

Continuously streams logs as they are generated.

```
$ docker logs -f CONTAINER
```

```
$ docker logs -f web
172.17.0.1 - - [10/Oct/2026:10:15:23]
"GET / HTTP/1.1" 200 615 "-" "curl/8.0"
172.17.0.1 - - [10/Oct/2026:10:15:24]
"GET /favicon.ico HTTP/1.1" 404 555 "-"
```

WHEN TO USE IT

- To watch logs live while troubleshooting.
- Useful during application startup or errors.

3 INSPECT A CONTAINER

Shows detailed configuration and metadata.

```
$ docker inspect CONTAINER
```

```
$ docker inspect web
[{
    "Id": "a1b2c3d4e5f6...",
    "Name": "/web",
    "State": { "Status": "running" },
    "Config": { "Image": "nginx:alpine" }
}]
```

WHEN TO USE IT

- To view IP address, mounts, environment variables, ports, image, and more.
- Useful for debugging and verification.

4 VIEW CONTAINER RESOURCE USAGE

Displays real-time CPU, memory, and I/O usage.

```
$ docker stats
```

CONTAINER	CPU %	MEM USAGE / LIMIT	NET I/O
web	0.12%	12.3MiB / 2GiB	1.2kB / 1.1kB
db	0.45%	78.1MiB / 2GiB	5.4kB / 2.3kB

WHEN TO USE IT

- To check if a container is using too much CPU or memory.
- Useful when containers are slow or unresponsive.

5 VIEW PROCESSES INSIDE A CONTAINER

Shows the running processes inside the container.

```
$ docker top CONTAINER
```

```
$ docker top web
UID      PID      PPID     CMD
root     1        0        nginx: master process
nginx    7        1        nginx: worker process
```

WHEN TO USE IT

- To see what processes are running inside a container.
- Useful for debugging unexpected behaviour.

6 VIEW CONTAINER PORT MAPPINGS

Shows which host ports are mapped to the container.

```
$ docker port CONTAINER
```

```
$ docker port web
80/tcp -> 0.0.0.0:8080
80/tcp -> [::]:8080
```

WHEN TO USE IT

- To verify port mappings.
- Useful when the application is not accessible.

7 TROUBLESHOOTING UNEXPECTEDLY EXITED CONTAINERS

A container may exit immediately due to errors in the application, configuration, or environment.

COMMON CAUSES

- Application error or crash
- Missing environment variables
- Wrong command or entrypoint
- Permission issues
- Port conflicts
- Container out of memory
- Misconfigured volumes

TROUBLESHOOTING STEPS

1	Check container status	`docker ps -a`
2	View logs	`docker logs CONTAINER`
3	Inspect configuration	`docker inspect CONTAINER`
4	Check resource usage	`docker stats`
5	Check processes	`docker top CONTAINER`
6	Verify ports	`docker port CONTAINER`
7	Review exit code	`docker inspect --format '{{.State.ExitCode}}' CONTAINER`

EXAMPLE

Container exits immediately:

```
$ docker ps -a
$ docker logs web
```

Check the exit code

```
$ docker inspect --format
'{{.State.ExitCode}}' web
```

JUNIOR ENGINEER TIP Always check the logs first. Most container problems are caused by application errors that are clearly shown in the logs.

DOCKER

EXECUTING COMMANDS INSIDE CONTAINERS

1 OPEN A SHELL INSIDE A CONTAINER

Start an interactive shell inside a running container.

```
$ docker exec -it CONTAINER sh
```

```
$ docker exec -it web sh
/ #
```

WHEN TO USE IT

- To run commands inside a running container.
- Useful for debugging and exploration.

2 VIEW ENVIRONMENT VARIABLES

Display environment variables inside a container.

```
$ docker exec CONTAINER env
```

```
$ docker exec web env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin
NGINX_VERSION=1.25.3
```

WHEN TO USE IT

- To check application configuration.
- Useful for verifying environment variables.

3 LIST PROCESSES INSIDE A CONTAINER

View running processes inside the container.

```
$ docker exec CONTAINER ps
```

```
$ docker exec web ps
PID   USER     TIME   COMMAND
  1   root     0:00   nginx: master process
  7   nginx    0:00   nginx: worker process
```

WHEN TO USE IT

- To see what processes are running.
- Useful for troubleshooting and verifying services.

4 COPY FILES TO AND FROM A CONTAINER

Copy files between your machine and a container.

Copy from host to container

```
$ docker cp file.txt web:/app/
```

Copy from container to host

```
$ docker cp web:/app/file.txt ./
```

WHEN TO USE IT

- To copy configuration files, logs or application files.
- Useful for debugging or moving files in and out.

5 SEE CHANGES TO THE CONTAINER FILESYSTEM

Shows changes made to files inside a container.

```
$ docker diff CONTAINER
```

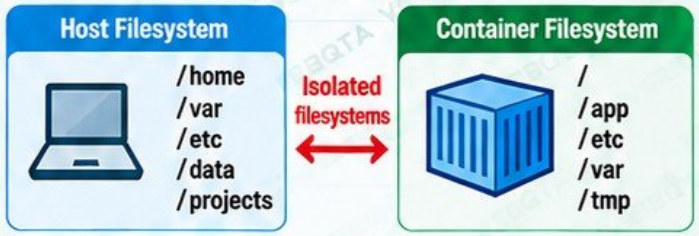
```
$ docker diff web
C /etc/nginx/nginx.conf
A /app
D /tmp/old-file.txt
```

WHEN TO USE IT

- To see which files have been changed, added or deleted.
- Useful for understanding what changed inside a container.

6 CONTAINER FILESYSTEM vs HOST FILESYSTEM

Understand the difference between the container's filesystem and your host's filesystem.



- A container has its own isolated filesystem.
- Changes inside the container do not affect your host unless you use volumes or bind mounts.

COMMON USE CASES

Debug a running application	`docker exec -it CONTAINER sh`
Check configuration	`docker exec CONTAINER env`
View running processes	`docker exec CONTAINER ps`
Copy files for debugging	`docker cp`
See file changes	`docker diff CONTAINER`

BEST PRACTICES

- Use read-only commands when possible.
- Avoid making unnecessary changes inside containers.
- Use volumes for persistent data.
- Always verify file paths.
- Use non-root users when possible.

COMMON MISTAKES

- Using docker exec on a stopped container.
- Forgetting the `-it` flag for an interactive shell.
- Copying files to the wrong path.
- Expecting container changes to appear on the host without volumes.
- Not checking file permissions.

JUNIOR ENGINEER TIP

Use `docker exec`, `docker cp` and `docker diff` to investigate containers instead of guessing. Always check the filesystem, logs and environment when troubleshooting.

DOCKER

DOCKER VOLUMES AND PERSISTENT DATA

1 CREATE A VOLUME

Creates a named volume for persistent storage.

```
$ docker volume create appdata
appdata
```

WHEN TO USE IT

- To create a named volume for storing application data.
- Useful for databases, application files, etc.

2 LIST VOLUMES

Shows all volumes on your machine.

```
$ docker volume ls
DRIVER    VOLUME NAME
local     appdata
local     dbdata
```

WHEN TO USE IT

- To see all existing volumes.
- Useful for checking volume names and cleaning up.

3 INSPECT A VOLUME

Shows detailed information about a volume.

```
$ docker volume inspect appdata
[
  {
    "Name": "appdata",
    "Driver": "local",
    "Mountpoint": "/var/lib/docker/volumes/appdata/_data",
    "Scope": "local"
  }
]
```

WHEN TO USE IT

- To view the volume's mount path, driver and details.
- Useful for troubleshooting and understanding where data is stored.

4 RUN A CONTAINER WITH A VOLUME

Mounts the named volume into a container.

```
$ docker run -v appdata:/data nginx:alpine
```



WHEN TO USE IT

- To store application data inside a volume instead of the container's writable layer.
- Useful for databases, CMS, and any data you need to keep after containers are deleted.

5 REMOVE A VOLUME

Deletes a volume (must not be in use).

```
$ docker volume rm appdata
appdata
```

WHEN TO USE IT

- To delete a volume you no longer need.
- Useful for cleaning up unused data and freeing disk space.

6 WHY PERSISTENT STORAGE MATTERS

Explains why volumes are important for real applications.

KEY BENEFITS

- Data survives container restarts.
- Data persists even if the container is deleted.
- Essential for databases (e.g. MySQL, PostgreSQL).
- Keeps application uploads, configs and user data safe.
- Separates data from containers, making it easier to manage and backup.
- Improves reliability in development, testing and production.

JUNIOR ENGINEER TIPS

- Use named volumes instead of anonymous volumes.
- Always verify data is stored in the volume with `docker volume inspect`.
- Backup important volumes, especially for databases.
- Don't delete a volume that is still in use.
- Use clear, meaningful names (e.g. appdata, dbdata).

COMMON USE CASES

- Databases (MySQL, PostgreSQL)
- Application files and uploads
- WordPress and other CMS data
- Configuration files
- Logs and monitoring data
- Development and testing environments

COMMON MISTAKES

- Using anonymous volumes and forgetting their names.
- Deleting a volume that is still in use.
- Expecting data to persist without a volume.
- Confusing host paths with named volumes.
- Not backing up important data.

DOCKER

BIND MOUNTS AND FILE SHARING

1 USE BIND MOUNTS WITH --MOUNT

Mounts a host directory or file into a container using the --mount flag.

```
$ docker run --mount type=bind,
source=/path/on/host,target=/app nginx:alpine
```



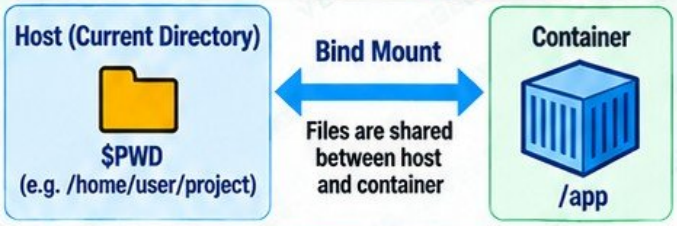
WHEN TO USE IT

- To share files or directories from your host into a container.
- Useful for development, configuration files, and static content.

2 USE BIND MOUNTS WITH -v (SHORT SYNTAX)

Mounts a host directory into a container using the -v flag.

```
$ docker run -v "$PWD":/app nginx:alpine
```



WHEN TO USE IT

- To quickly mount your current directory.
- Useful for development and testing.

3 READ-ONLY BIND MOUNTS

Mounts a host directory or file as read-only.

```
$ docker run -v "$PWD":/app:ro nginx:alpine
```



WHEN TO USE IT

- To prevent accidental changes to host files.
- Useful for configuration files, scripts, or sensitive data.

4 HOST PATHS VERSUS CONTAINER PATHS

Host paths refer to locations on your machine.
Container paths refer to locations inside the container.

Host Path (Example)	Container Path (Example)
/home/user/project C:\Users\user\project /var/data	/app /data /etc/nginx /var/www

- The host path must exist (for -v) or be accessible (for --mount).
- The container path is where the files appear inside the container.
- You can map any host path to any container path.
- Use absolute paths for clarity and to avoid errors.

5 BIND MOUNT PERMISSIONS

Files keep the host's permissions when mounted into a container.

```
$ ls -l /home/user/project
$ docker run -v /home/user/project:/app nginx:alpine
```

Key Points

- The container sees the same file permissions as the host.
- The user inside the container must have permission to read/write the mounted files.
- You may need to adjust permissions using chmod or chown on the host.
- On Linux, file ownership and permissions matter.
- On Windows/macOS (with Docker Desktop), permissionons are handled differently.

6 WHY BIND MOUNTS REQUIRE SECURITY AWARENESS

Bind mounts give a container access to your host's filesystem, which can be a security risk if not used carefully.

SECURITY RISKS

- Container can read or modify important host files.
- Accidental exposure of sensitive data (e.g. SSH keys, credentials).
- Malicious containers can access host files.
- Mounting the entire filesystem (e.g. /) is dangerous.
- Misconfigured permissions can lead to unauthorized access.

BEST PRACTICES

- Only mount the specific directory or file you need.
- Use read-only mounts (:ro) when possible.
- Avoid mounting sensitive directories (e.g. /home, /etc).
- Use non-root users in containers.
- Regularly review what is mounted into containers.

JUNIOR ENGINEER TIPS

- Always use absolute paths for bind mounts.
- Use read-only mounts for configuration files.
- Check permissions if you get "Permission denied" errors.
- Avoid mounting sensitive directories.

COMMON USE CASES

- Development (mount source code).
- Configuration files (nginx.conf, app settings).
- Static website content.
- Sharing scripts and data for testing.
- Persisting logs during development.

COMMON MISTAKES

- Using relative paths that don't exist.
- Forgetting to create the host directory.
- Mounting the entire filesystem (/).
- Not using read-only when appropriate.
- Running containers as root with host mounts.

DOCKER

DOCKER NETWORKING COMMANDS

1 LIST NETWORKS

Shows all networks on your machine.

```
$ docker network ls
```

NETWORK ID	NAME	DRIVER	SCOPE
a1b2c3d4e5f6	bridge	bridge	local
b2c3d4e5f6g7	host	host	local
c3d4e5f6g7h8	none	null	local
d4e5f6g7h8i9	appnet	bridge	local

WHEN TO USE IT

- To see all existing networks.
- Useful for checking network names and drivers.

2 CREATE A NETWORK

Creates a new custom network.

```
$ docker network create appnet
```



WHEN TO USE IT

- To create an isolated network for your applications.
- Useful for container-to-container communication.

3 INSPECT A NETWORK

Shows detailed information about a network.

```
$ docker network inspect appnet
```

```
[
  {
      "Name": "appnet",
      "Id": "d4e5f6g7h8i9...",
      "Driver": "bridge",
      "Scope": "local",
      "Containers": { ... }
  }
]
```

WHEN TO USE IT

- To view network configuration, subnet, gateway and connected containers.
- Useful for troubleshooting network issues.

4 CONNECT A CONTAINER TO A NETWORK

Connects a running container to a network.

```
$ docker network connect appnet web
```



WHEN TO USE IT

- To add an existing container to a network.
- Useful when you want containers to communicate.

5 DISCONNECT A CONTAINER FROM A NETWORK

Removes a container from a network.

```
$ docker network disconnect appnet web
```

WHEN TO USE IT

- To remove a container from a network.
- Useful for isolation or troubleshooting.

6 REMOVE A NETWORK

Deletes a network (must not be in use).

```
$ docker network rm appnet
```

WHEN TO USE IT

- To delete a network you no longer need.
- Useful for cleaning up unused networks.

CONTAINER-TO-CONTAINER COMMUNICATION

Containers on the same user-defined network can communicate using container names.



- Use container names instead of IP addresses.
- No need for port mapping between containers.
- Works automatically on user-defined networks.
- Useful for multi-container applications (e.g. web + database).

COMMON NETWORK DRIVERS

Driver	Description
bridge	Default driver. Isolates containers on the same host.
host	Uses the host's network stack (no isolation).
none	No networking (fully isolated).
overlay	Used for multi-host networking (in Docker Swarm).
macvlan	Assigns a MAC address to containers (advanced use).

BEST PRACTICES

- Use user-defined networks instead of the default bridge.
- Use container names for communication.
- Keep networks organized and use meaningful names.
- Remove unused networks.
- Use network inspection for troubleshooting.

JUNIOR ENGINEER TIP

- Use `docker network inspect` to understand a network's IP range, connected
- containers, and driver. This helps you debug communication issues quickly.

COMMON MISTAKES

- Trying to remove a network that is still in use.
- Using IP addresses instead of container names.
- Not using a user-defined network for multi-container apps.
- Misunderstanding the difference between exposed ports and networking.

DOCKER

PORTS AND CONNECTIVITY TROUBLESHOOTING

1 RUN A CONTAINER WITH PORT MAPPING

Maps a host port to a container port.

```
$ docker run -p 127.0.0.1:8080:80 \
--name web nginx:alpine
```

WHEN TO USE IT

- To make a containerized service accessible from your host.
- Useful for testing web applications, APIs, etc.

2 VIEW MAPPED PORTS

Shows which ports are mapped to the container.

```
$ docker port CONTAINER
```

```
$ docker port web
80/tcp -> 127.0.0.1:8080
80/tcp -> [::1]:8080
```

WHEN TO USE IT

- To check the host ports mapped to container ports.
- Useful when you don't remember the port mapping.

3 INSPECT NETWORK AND PORT CONFIGURATION

Shows detailed networking information.

```
$ docker inspect CONTAINER
```

```
$ docker inspect web
  "NetworkSettings": {
    "Ports": {
      "80/tcp": [
        {
          "HostIp": "127.0.0.1",
          "HostPort": "8080"
        }
    ] ]
  }
```

WHEN TO USE IT

- To view detailed port mapping and network settings.
- Useful for troubleshooting connectivity issues.

4 TEST CONNECTIVITY WITH CURL

Sends a request to the service running in the container.

```
$ curl http://localhost:8080
```

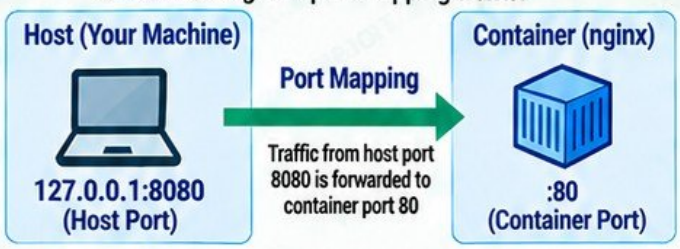
```
<!DOCTYPE html>
<html>
<head><title>Welcome to nginx!</title></head>
<body>
<h1>Welcome to nginx!</h1>
</body>
</html>
```

WHEN TO USE IT

- To verify that the service is reachable.
- Useful for testing web servers and APIs.

5 HOST PORTS VERSUS CONTAINER PORTS

Understanding how port mapping works.



- The host port is the port on your machine that you access.
- The container port is the port inside the container where the service runs.
- You can map to a specific IP (e.g. 127.0.0.1) or to all interfaces (e.g. 0.0.0.0).

6 DIAGNOSING CONNECTION REFUSED AND PORT CONFLICTS

Common issues and how to fix them.

COMMON ISSUES	CAUSE AND SOLUTION
Connection refused	• Container is not running. • Wrong port mapping. • Service inside container is not listening. • Check with `docker ps`, `docker logs` and `docker port`.
Port already in use	• Another process is using the host port. • Find the process: `lsof -i :8080` or `netstat -tulpn | grep 8080` • Use a different host port.
Cannot access from outside	• You mapped to `127.0.0.1` (only local access). • Use `-p 0.0.0.0:8080:80` to allow external access. • Check firewall settings.
Application not reachable	• Container exited or crashed. • Wrong container port. • Service not started inside container. • Check with `docker ps`, `docker logs` and `docker inspect`.

JUNIOR ENGINEER TIPS

- Always verify the container is running (`docker ps`).
- Use `docker port` or `docker inspect` to confirm port mappings.
- Test with curl before assuming the application is down.
- Use specific host IPs (127.0.0.1) for local-only access.
- Be aware of firewall rules on your host or cloud server.

USEFUL TROUBLESHOOTING COMMANDS

`docker ps -a`	List all containers
`docker logs CONTAINER`	View container logs
`docker port CONTAINER`	Show port mappings
`docker inspect CONTAINER`	Inspect configuration
`lsof -i :PORT`	Check what's using a port
`netstat -tulpn | grep PORT`	Find listening processes
`curl http://localhost:PORT`	Test connectivity

BEST PRACTICES

- Use meaningful port mappings.
- Document which ports are used.
- Avoid exposing unnecessary ports.
- Use firewalls and security groups.
- Regularly check for port conflicts.
- Verify application health after startup.

DOCKER

DOCKERFILE INSTRUCTIONS CHEAT SHEET

1 FROM

Specifies the base image for your container.

```
$ FROM ubuntu:24.04
```

WHEN TO USE IT

- To set the starting point for your image.
- Use official and trusted base images.

2 WORKDIR

Sets the working directory inside the container.

```
$ WORKDIR /app
```

WHEN TO USE IT

- To specify the directory for subsequent commands.
- Creates the directory if it does not exist.

3 COPY

Copies files from your local machine into the container image.

```
$ COPY . /app
```

WHEN TO USE IT

- To copy application code, configs or other files.
- Use .dockerignore to exclude unnecessary files.

4 RUN

Runs a command during the image build process.

```
$ RUN apt-get update && \
      apt-get install -y nginx
```

WHEN TO USE IT

- To install packages and run setup commands.
- Each RUN command creates a new image layer.

5 CMD

Specifies the default command to run when a container starts.

```
$ CMD ["nginx", "-g",
      "daemon off;"]
```

WHEN TO USE IT

- To set the default command.
- Can be overridden at runtime using `docker run`.

6 ENTRYPOINT

Defines the main executable for the container.

```
$ ENTRYPOINT ["nginx", "-g",
              "daemon off;"]
```

WHEN TO USE IT

- To set the main command that always runs.
- Often used for applications that must always start the same way.

7 EXPOSE

Documents the port that the container listens on.

```
$ EXPOSE 80
```

WHEN TO USE IT

- To show which port the application uses.
- Does not publish the port automatically (use -p with `docker run`).

8 USER

Sets the user to run commands and the container.

```
$ USER appuser
```

WHEN TO USE IT

- To run the container as a non-root user.
- Improves security by limiting permissions.

SAMPLE DOCKERFILE

```
FROM ubuntu:24.04
WORKDIR /app
RUN apt-get update && \
    apt-get install -y nginx
COPY . /app
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

HOW DOCKERFILES BECOME CONTAINER IMAGES

A Dockerfile is a set of instructions. When you run **docker build**, Docker reads the instructions, executes them step by step, and creates a container image with multiple layers.



BEST PRACTICES

- Use official base images.
- Keep images small and secure.
- Use .dockerignore.
- Combine related RUN commands.
- Use a non-root user.
- Set a clear working directory.
- Document exposed ports.
- Avoid unnecessary files.
- Follow security best practices.

COMMON MISTAKES

- Using untrusted base images.
- Copying unnecessary files (e.g. .git, node_modules).
- Not using .dockerignore.
- Running as root.
- Creating too many layers.
- Not specifying a WORKDIR.
- Exposing ports but not publishing them with -p.
- Storing secrets in the Dockerfile.
- Using outdated packages.

DOCKER

BUILDING AND TAGGING DOCKER IMAGES

1 BUILD AN IMAGE

Builds a Docker image from the Dockerfile in the current directory.

```
$ docker build -t myapp:1.0 .
```

WHEN TO USE IT

- To build a new image from a Dockerfile.
- -t sets the image name and tag.
- . specifies the build context (current directory).

2 BUILD WITHOUT CACHE

Ignores the build cache and rebuilds all layers.

```
$ docker build --no-cache -t myapp:1.0 .
```

WHEN TO USE IT

- To ensure a clean build.
- Useful when you suspect cached layers are causing issues or you want the latest packages.

3 PULL LATEST BASE IMAGES

Always pulls the latest version of base images during the build.

```
$ docker build --pull -t myapp:1.0 .
```

WHEN TO USE IT

- To make sure you are using the latest base image.
- Useful for security updates and getting latest fixes.

4 TAG AN EXISTING IMAGE

Creates a new tag for an existing image.

```
$ docker image tag myapp:1.0 myapp:latest
```



WHEN TO USE IT

- To create additional tags (e.g. latest, version numbers).
- Useful for organizing and pushing images to registries.

5 VIEW IMAGE BUILD HISTORY

Shows the history of layers in an image.

```
$ docker image history myapp:1.0
```

IMAGE	CREATED	CREATED BY	SIZE
a1b2c3d4	2 hours ago	/bin/sh -c …	120MB
d4e5f6g7	2 hours ago	COPY . /app	5MB
h7i8j9k0	2 hours ago	RUN apt-get …	200MB
l1m2n3o4	3 hours ago	FROM ubuntu:24.04	77MB

WHEN TO USE IT

- To see how an image was built and what each layer did.
- Useful for troubleshooting and image size optimization.

6 BUILD CONTEXT, CACHING AND TAGGING MISTAKES

Common issues when building and tagging images.

BUILD CONTEXT

- The build context is the set of files sent to the Docker daemon.
- Use . to send the current directory.
- Avoid large or unnecessary files. Use .dockerignore.

CACHING

- Docker caches layers to speed up builds.
- If a layer hasn't changed, it is reused.
- Changes in earlier layers can invalidate later layers.
- Use --no-cache when you need ta completely fresh build.

TAGGING MISTAKES

- Not tagging the image leads to <none>:<none>.
- Using inconsistent tags (e.g. latest in production).
- Forgetting to tag images before pushing to a registry.
- Overwriting important tags.

JUNIOR ENGINEER TIPS

- Always use a .dockerignore file.
- Keep the build context small.
- Tag images with meaningful names and versions.
- Use --pull to get the latest base images.
- Check the build output for errors carefully.

COMMON USE CASES

- Building your application image.
- Creating versioned tags (e.g. 1.0, 1.1).
- Rebuilding without cache for a clean build.
- Keeping base images up to date.
- Inspecting image history to reduce size.

BEST PRACTICES

- Use specific and meaningful tags.
- Combine --pull with regular builds.
- Use .dockerignore to exclude unnecessary files.
- Keep images small and secure.
- Avoid using latest in production.

DOCKER

IMAGE REGISTRIES AND DISTRIBUTION

1 LOGIN TO A REGISTRY

Authenticates your Docker client with a registry (e.g. Docker Hub or a private registry).

```
$ docker login
```

Examples:

```
$ docker login
$ docker login myregistry.com
```

WHEN TO USE IT

- When you need to pull or push images from a private or authenticated registry.
- Use on Docker Hub, AWS ECR, GHCR, or internal registries.

2 LOGOUT FROM A REGISTRY

Removes your authentication credentials for a registry.

```
$ docker logout
```

Examples:

```
$ docker logout
$ docker logout myregistry.com
```

WHEN TO USE IT

- To remove saved credentials from your machine.
- Useful for security, especially on shared systems.

3 TAG AN IMAGE

Creates a new tag for an existing image.

```
$ docker tag
```

Example:

```
$ docker tag myapp:1.0
  myusername/myapp:1.0
```

WHEN TO USE IT

- To give an image a new name or tag before pushing to a registry.
- Useful for versioning and organizing images.

4 PUSH AN IMAGE

Uploads your image to a registry.

```
$ docker push
```

Example:

```
$ docker push myusername/myapp:1.0
```

WHEN TO USE IT

- To share your image with others.
- To deploy images to a remote registry (Docker Hub, AWS ECR, etc.).

5 PULL AN IMAGE

Downloads an image from a registry to your local machine.

```
$ docker pull
```

Example:

```
$ docker pull nginx:latest
$ docker pull myregistry.com/myapp:1.0
```

WHEN TO USE IT

- To get an image from a registry.
- Useful for running official images or images shared by your team.

6 INSPECT AN IMAGE

Shows detailed information about an image.

```
$ docker image inspect
```

Example:

```
$ docker image inspect
  nginx:latest
```

WHEN TO USE IT

- To view detailed imetadata about the image.
- Useful for troubleshooting and verifying image details.

HOW IMAGE DISTRIBUTION WORKS



PRIVATE REGISTRIES AND CREDENTIAL PROTECTION

Private registries allow you to store your organization's images securely. Always protect your credentials.

EXAMPLES OF REGISTRIES

- Docker Hub (public)
- AWS ECR (private)
- GitHub Container Registry (GHCR)
- Self-hosted registry (e.g. Harbor, Nexus, etc.)

SECURITY BEST PRACTICES

- Use strong passwords or token-based authentication.
- Use short-lived access tokens.
- Store credentials securely (e.g. Docker credential helper).
- Do not hardcode credentials in Dockerfiles.
- Use private registries for sensitive or internal images.
- Regularly rotate credentials.

COMMON USE CASES

- Sharing images with your team.
- Deploying applications to cloud platforms.
- Distributing base images for standardized environments.
- Using private registries for internal applications.
- Pulling official images for development and testing.

COMMON MISTAKES

- Forgetting to tag the image before pushing.
- Pushing to the wrong repository.
- Using latest tag only (no versioning).
- Exposing credentials in Dockerfiles.
- Not using a .dockerignore file.
- Using public registries for sensitive images.

JUNIOR ENGINEER TIPS

- Always tag images with meaningful names and versions.
- Use `docker login` and `docker logout` securely.
- Use environment variables or credential helpers for private registries.
- Verify the image with `docker image inspect`.
- Keep sensitive images in private registries.

DOCKER

DOCKER COMPOSE ESSENTIAL COMMANDS

1 CHECK DOCKER COMPOSE VERSION

Shows the installed Docker Compose version.

```
$ docker compose version
```

```
Docker Compose version v2.24.1
```

WHEN TO USE IT

- To confirm Docker Compose is installed.
- Useful for troubleshooting version issues.

2 VALIDATE AND VIEW CONFIGURATION

Parses and shows the final configuration from your docker-compose.yml file.

```
$ docker compose config
```

Example output (partial):

```
services:
  web:
    image: nginx:latest
    ports:
      - "8080:80"
```

WHEN TO USE IT

- To validate your docker-compose.yml file.
- To see the final, merged configuration.
- Useful for debugging configuration issues.

3 START SERVICES

Builds (if needed) and starts all services defined in docker-compose.yml.

```
$ docker compose up -d
```

```
Creating network "myapp_default" ... done
Creating db      ... done
Creating web     ... done
```

WHEN TO USE IT

- To start a multi-container application.
- Use -d to run in detached mode (in background).
- Uuilds images if they don't exist.

4 VIEW RUNNING SERVICES

Shows the status of all services in your application.

```
$ docker compose ps
```

NAME	COMMAND	STATE	PORTS
myapp-web	"nginx -g..."	running	0.0.0.0:8080->80/tcp
myapp-db	"docker-entrypoint..."	running	3306/tcp

WHEN TO USE IT

- To check which containers are running.
- To view their status, ports and names.
- Useful for monitoring your application.

5 VIEW SERVICE LOGS

Shows logs from your services.

```
$ docker compose logs
```

```
web  | /docker-entrypoint.sh: Configuration
web  | 2026-10-12 10:15:23 [notice]
db   | 2026-10-12 10:15:24 [Note] Server started
db   | 2026-10-12 10:15:25 [Note] Ready for connections
```

WHEN TO USE IT

- To view logs from all services.
- Add -f to follow logs in real time (docker compose logs -f).
- Useful for debugging application issues.

6 STOP AND REMOVE SERVICES

Stops and removes containers, networks and other resources created by docker compose.

```
$ docker compose down
```

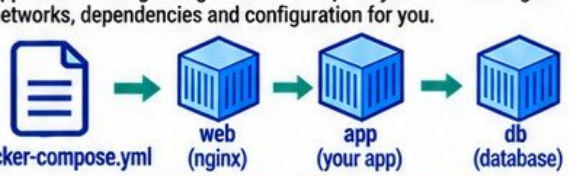
```
Stopping myapp-web  ... done
Stopping myapp-db   ... done
Removing myapp-web  ... done
Removing myapp-db   ... done
Removing network myapp_default   ... done
```

WHEN TO USE IT

- To stop and clean up your application.
- Removes containers, networks and volumes created by Compose.
- Useful when you no longer need the application running.

UNDERSTANDING MULTI-CONTAINER APPLICATIONS

Docker Compose allows you to define and run multi-container applications using a single docker-compose.yml file. It manages networks, dependencies and configuration for you.



COMMON USE CASES

- Running web applications (web + db).
- Starting multiple dependent services.
- Managing development environments.
- Testing full application stacks.
- Simplifying container orchestration.
- Sharing consistent environments with your team.

BEST PRACTICES

- Keep your docker-compose.yml versioned in Git.
- Use meaningful service names.
- Use environment variables for configuration.
- Use volumes for persistent data.
- Check logs when troubleshooting.
- Use docker compose config to validate your file.

DOCKER

COMPOSE OPERATIONS AND DEBUGGING

1 BUILD SERVICES

Builds or rebuilds images for services defined in docker-compose.yml.

```
$ docker compose build
```

```
Building web
Building db
Successfully built
```

WHEN TO USE IT

- When you have changed the Dockerfile or application code.
- To rebuild one or more services.
- Useful during development and troubleshooting.

2 PULL LATEST IMAGES

Pulls the latest images for services from a registry (e.g. Docker Hub).

```
$ docker compose pull
```

```
Pulling web    ... done
Pulling db     ... done
Pulling redis  ... done
```

WHEN TO USE IT

- To get the latest version of images.
- When your services use official images from a registry.
- Useful to keep your environment up to date.

3 EXECUTE A COMMAND IN A RUNNING SERVICE

Runs a command inside a running service container.

```
$ docker compose exec web sh
```

```
/app # ls
Dockerfile   requirements.txt   app.py
```

WHEN TO USE IT

- To get a shell inside a running container.
- To run administrative or debugging commands.
- Useful for checking files, environment variables, etc.

4 RUN A ONE-OFF COMMAND

Runs a command in a temporary container for a service.

```
$ docker compose run --rm web sh
```

```
Creating myapp_web_run ... done
/app # python manage.py migrate
```

WHEN TO USE IT

- To run one-off tasks (e.g. database migrations).
- Runs a temporary container and removes it after exit (--rm).
- Useful for administrative and maintenance tasks.

5 RESTART SERVICES

Restarts one or more services.

```
$ docker compose restart
```

```
Restarting myapp-web  ... done
Restarting myapp-db   ... done
```

WHEN TO USE IT

- To apply configuration changes.
- To recover from temporary issues.
- Useful when a service is unhealthy or stuck.

6 FOLLOW LOGS IN REAL TIME

Shows logs from services and follows new log output.

```
$ docker compose logs -f
```

```
web    | Starting application...
web    | Connected to database
web    | Listening on port 8000
db     | database system is ready to accept connections
```

WHEN TO USE IT

- To monitor logs live while troubleshooting. Add -f to follow logs in real time.
- Useful for diagnosing startup issues and runtime errors.

DIAGNOSING APPLICATION STARTUP FAILURES

- Check logs: `docker compose logs -f`
- Verify the application configuration and environment variables.
- Ensure dependent services (e.g. database) are running and healthy.
- Check if the application is listening on the correct port.
- Look for errors in application logs.

DIAGNOSING DATABASE STARTUP FAILURES

- Check database logs: `docker compose logs -f db`
- Verify environment variables (e.g. passwords).
- Ensure required directories and volumes exist.
- Check if the database port is correctly exposed.
- Look for initialization errors or permission issues.
- Ensure the database has enough resources (CPU, memory, disk space).

COMMON ISSUES AND SOLUTIONS

ISSUE	SOLUTION
Service fails to start	Check logs, configuration and environment variables.
Database connection refused	Ensure the database is running and accessible.
Service keeps restarting	Check logs for errors and fix the root cause.
Image not found	Run `docker compose pull` or build the image.
Permission errors	Check file permissions and volume settings.

DOCKER

RESOURCE MANAGEMENT

1 VIEW RESOURCE USAGE

Shows real-time resource usage for running containers.

```
$ docker stats
```

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O
a1b2c3d4e5f6	web	0.52%	128MiB / 1GiB	12.5%	1.2MB / 1.1MB
b2c3d4e5f6g7	db	1.23%	256MiB / 2GiB	12.5%	2.3MB / 2.1MB

WHEN TO USE IT

- To monitor CPU, memory, network and I/O usage.
- To identify containers using too many resources.
- Useful for performance tuning and troubleshooting.

2 SET MEMORY LIMITS

Limits the amount of memory a container can use.

```
$ docker run --memory 512m nginx
```

Examples:

```
$ docker run --memory 512m nginx
$ docker run --memory 1g --memory-swap 1g nginx
```

WHEN TO USE IT

- To prevent a container from using too much memory.
- To keep your system stable.
- Useful in production and shared environments.

3 SET CPU LIMITS

Limits the amount of CPU resources a container can use.

```
$ docker run --cpus "0.5" nginx
```

Examples:

```
$ docker run --cpus "0.5" nginx
$ docker run --cpus "1.5" nginx
```

WHEN TO USE IT

- To restrict how much CPU a container can use.
- To prevent one container from affecting others.
- Useful on shared hosts and multi-tenant systems.

4 UPDATE CONTAINER RESOURCES

Updates resource limits for a running container.

```
$ docker update --memory 1g --cpus 1 myapp
```

WHEN TO USE IT

- To change resource limits without stopping the container.
- To respond to increased or decreased workload.
- Useful for adjusting running containers in real time.

5 INSPECT CONTAINER CONFIGURATION

Shows detailed information about a container, including resource settings.

```
$ docker inspect myapp
```

```
"HostConfig": {
    "Memory": 536870912,
    "CpuQuota": 50000,
    "CpuPeriod": 100000,
    "NanoCpus": 500000000,
}   ...
```

WHEN TO USE IT

- To view the current resource limits and settings.
- To verify how a container was configured.
- Useful for debugging and auditing.

6 MEMORY LIMITS, CPU LIMITS AND OOM KILLS

When a container exceeds its memory limit, the kernel may kill it (OOM kill).

COMMON SCENARIOS

Container uses more memory than the specified limit.	Kernel triggers an OOM kill.
Container uses more CPU than allowed.	CPU is throttled, not killed.
You may see errors in logs like:	`oom-kill: constraint=CONSTRAINT_MEMCG`

HOW TO PREVENT OOM KILLS

- Set appropriate memory limits (`--memory`).
- Monitor usage with `docker stats`.
- Optimize your application's memory usage.
- Use resource limits that match your application needs.
- Check logs to identify memory-related issues.

COMMON MISTAKES

- Not setting resource limits.
- Giving containers more resources than needed.
- Ignoring memory usage and OOM kill errors.
- Assuming CPU limits guarantee performance.

BEST PRACTICES

- Set reasonable memory and CPU limits.
- Monitor resource usage regularly.
- Use `docker stats` and `docker inspect`.
- Adjust limits based on real application needs.
- Plan for spikes in resource usage.

REAL-WORLD USE CASES

- Preventing a misbehaving container from consuming all system resources.
- Running multiple containers on a shared server.
- Keeping production environments stable.
- Optimizing performance and cost.
- Troubleshooting slow or crashing containers.

DOCKER

SECURITY COMMANDS AND PRACTICES

1 RUN A CONTAINER IN READ-ONLY MODE

Mounts the container's root filesystem as read-only.

```
$ docker run --read-only nginx
```

WHEN TO USE IT

- To prevent accidental or malicious file changes.
- To reduce the impact of a container breakout.
- Useful for more secure and immutable containers.

2 RUN A CONTAINER AS A NON-ROOT USER

Runs the container process as a specific user.

```
$ docker run --user 1000:1000 nginx
```

Examples:

```
$ docker run --user 1000:1000 nginx  # UID:GID
$ docker run --user nobody nginx     # named user
```

WHEN TO USE IT

- To apply the principle of least privilege.
- To reduce the risk of container escape.
- Useful for running applications without root access.

3 DROP ALL LINUX CAPABILITIES

Removes all Linux capabilities from the container.

```
$ docker run --cap-drop ALL nginx
```

Examples:

```
$ docker run --cap-drop ALL nginx
$ docker run --cap-drop ALL --cap-add NET_BIND_SERVICE nginx
```

WHEN TO USE IT

- To minimize the powers of the container.
- To prevent unnecessary system access.
- Useful for running simple applications with limited needs.

4 PREVENT NEW PRIVILEGES

Stops processes in the container from gaining more privileges.

```
$ docker run --security-opt no-new-privileges nginx
```

WHEN TO USE IT

- To prevent privilege escalation inside the container.
- To add an extra layer of security.
- Useful for running untrusted applications.

5 INSPECT IMAGE SECURITY SETTINGS

Shows detailed configuration and security-related information about an image.

```
$ docker image inspect nginx
```

```
"Config": {
    "User": "nginx",
    "ExposedPorts": {
        "80/tcp": {}
    },
    "Labels": {
        "maintainer": "NGINX Docker Image"
    }
}
```

WHEN TO USE IT

- To check the default user, environment and capabilities.
- To review security-related settings.
- Useful for auditing images before using them.

LEAST PRIVILEGE AND CONTAINER ISOLATION

Run containers with only the permissions they need. Reduce the container's access to the host and other resources.

Use non-root users	Drop unnecessary capabilities	Prevent privilege escalation	Use read-only filesystems

ADDITIONAL SECURITY PRACTICES

- Use minimal base images (e.g. alpine).
- Keep images updated regularly.
- Scan images for vulnerabilities.
- Avoid running privileged containers.
- Use read-only root filesystem where possible.
- Limit container resources (CPU and memory).
- Use network policies and isolate containers.
- Only allow required capabilities.

COMMON MISTAKES

- Running containers as root.
- Giving unnecessary capabilities.
- Using privileged mode (--privileged).
- Not updating base images.
- Building images from untrusted sources.

REAL-WORLD USE CASES

- Running web applications more securely.
- Isolating untrusted or third-party applications.
- Restricting containers on shared servers.
- Meeting security and compliance requirements.
- Reducing the impact of container vulnerabilities.

TIPS FOR JUNIOR ENGINEERS

- Always use a non-root user.
- Remove unusted Linux capabilities.
- Use ---read-only when possible.
- Check images with `docker image inspect`.
- Combine multiple security options.
- Test your containers with minimal permissions.

DOCKER

HEALTH CHECKS AND MONITORING

1 INSPECT A CONTAINER WITH FORMATTING

Shows detailed information about a container using a custom format.

```
$ docker inspect --format \
'{{.State.Status}}' myapp
```

Examples:

```
$ docker inspect --format '{{.State.Status}}' myapp
$ docker inspect --format '{{.NetworkSettings.IPAddress}}'myapp
```

WHEN TO USE IT

- To get specific information about a container.
- To use in scripts and automation.
- Useful for monitoring, troubleshooting and reporting.

2 LIST RUNNING CONTAINERS

Shows current containers and their status.

```
$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	STATUS	PORTS	NAMES
a1b2c3d4e5f6	nginx	"nginx -g..."	Up 2 hours	80/tcp	web
f6e5d4c3b2a1	postgres	"docker-entrypoint..."	Up 3 hours	5432/tcp	db

WHEN TO USE IT

- To see which containers are running.
- To check container status, ports and names.
- Useful for quick health checks and troubleshooting.

3 MONITOR DOCKER EVENTS

Shows real-time events from the Docker daemon.

```
$ docker events
```

TIME	TYPE	ACTION	ACTOR
2026-10-12 10:15:01	container	start	myapp
2026-10-12 10:15:05	container	health_status	myapp
2026-10-12 10:16:12	container	stop	db

WHEN TO USE IT

- To monitor real-time container activity.
- To diagnose unexpected restarts or failures.
- Useful for understanding what is happening on your host.

4 VIEW RESOURCE USAGE

Shows real-time CPU, memory, network and I/O usage.

```
$ docker stats
```

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O
a1b2c3d4e5f6	web	0.52%	128MiB / 1GiB	12.5%	1.2MB / 1.1MB
f6e5d4c3b2a1	db	1.23%	256MiB / 2GiB	12.5%	2.3MB / 2.1MB

WHEN TO USE IT

- To monitor CPU, memory, network and I/O usage.
- To identify containers using too many resources.
- Useful for performance tuning and troubleshooting.

5 VIEW CONTAINER LOGS FROM A SPECIFIC TIME

Shows logs from a container with a time filter.

```
$ docker logs --since 10m myapp
```

```
2026-10-12T10:05:12Z Starting application...
2026-10-12T10:05:15Z Connected to database
2026-10-12T10:06:01Z Processing request...
2026-10-12T10:06:30Z Request completed
```

WHEN TO USE IT

- To view recent logs only.
- To troubleshoot application issues.
- Useful for finding errors without too much log output.

6 HEALTHCHECK DOCKERFILE INSTRUCTION

Defines a command to check if a container is healthy.

```
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD curl -f http://localhost:80/ || exit 1
```

WHEN TO USE IT

- To let Docker automatically check if your application is healthy.
- To detect unhealthy containers.
- Useful for reliable deployments and self-healing systems.

HEALTHY VERSUS RUNNING CONTAINERS

RUNNING

- The container process is running.
- Does not mean the application inside is healthy.
- Container can be running but the application may be broken.

HEALTHY

- The container has passed the health check defined in Dockerfile or at runtime.
- Indicates the application inside is working correctly.
- Use HEALTHCHECK to enable this.

COMMON ISSUES

- Container is running but application is unhealthy.
- Health check command fails.
- Missing or incorrect health check URL.
- High resource usage (CPU/memory).
- Not checking logs for errors.

BEST PRACTICES

- Use a meaningful health check command.
- Set appropriate interval, timeout and retries.
- Monitor logs regularly.
- Set resource limits to prevent instability.
- Use docker events for real-time visibility.

REAL-WORLD USE CASES

- Monitoring web application health.
- Detecting database connection issues.
- Auto-restarting unhealthy containers.
- Observing resource usage in production.
- Troubleshooting application failures.

DOCKER

CLEANUP AND DISK MANAGEMENT

1 DOCKER SYSTEM DF

Check disk usage of Docker resources.

```
$ docker system df
```

TYPE	TOTAL	ACTIVE	SIZE	RECLAIMABLE
Containers	4	2	1.2GB	500MB (42%)
Images	12	6	3.8GB	1.1GB (29%)
Volumes	5	3	2.1GB	600MB (29%)
Build Cache	8	5	1.5GB	800MB (53%)

WHEN TO USE IT

- To see how much disk space Docker is using.
- To find what is taking up the most space.
- To plan cleanup and storage management.

2 DOCKER CONTAINER PRUNE

Remove stopped containers.

```
$ docker container prune
```

This removes all stopped containers.
You will be asked to confirm.

WHEN TO USE IT

- To clean up stopped containers.
- To free up disk space.
- To keep only running containers.

3 DOCKER IMAGE PRUNE

Remove unused images.

```
$ docker image prune
```

This removes dangling (untagged) images.
You can use `-a` to remove all unused images.

WHEN TO USE IT

- To delete images that are no longer used.
- To free up disk space occupied by images.
- To keep only necessary images.

4 DOCKER BUILDER PRUNE

Clean up unused build cache.

```
$ docker builder prune
```

This removes unused build cache layers.
You can use `-a` to remove all unreferenced cache.

WHEN TO USE IT

- To clear build cache that is no longer needed.
- To reduce disk space used by builds.
- To keep your build environment clean.

5 DOCKER VOLUME PRUNE

Remove unused volumes.

```
$ docker volume prune
```

This removes all unused volumes
(not attached to any container).

WHEN TO USE IT

- To delete orphaned volumes.
- To free up disk space from unused data.
- To maintain a clean Docker environment.

6 DOCKER SYSTEM PRUNE

Clean up everything (containers, images, networks, volumes).

```
$ docker system prune
```

This removes stopped containers, dangling images, unused networks and volumes.
You can use `-a` to remove all unused images.

WHEN TO USE IT

- To perform a comprehensive cleanup.
- To free up significant disk space.
- To keep your system tidy and efficient.

PREVENTING ACCIDENTAL DELETION OF VALUABLE DATA

- Always check what will be deleted before running prune commands.
- Use filters (e.g. `--filter`) when possible.
- Avoid using `docker system prune -a` unless you really want to remove all unused images.
- Keep important data in volumes and back them up.
- Document and label important resources.

KEY TAKEAWAYS

- Use `docker system df` to understand disk usage.
- Clean up unused containers, images, volumes and build cache regularly.
- Use specific prune commands instead of `system prune` when possible.
- Always be careful to avoid deleting valuable data.
- A clean Docker environment is more efficient, secure and easier to maintain.

DOCKER

PRODUCTION TROUBLESHOOTING CHEAT SHEET

1 CONTAINER EXITS IMMEDIATELY

The container starts and then stops right away.

COMMON CAUSES

- Application error or misconfiguration
- Missing environment variables
- Wrong command or entrypoint

DIAGNOSTIC COMMANDS

```
docker ps -a
docker logs <container>
docker inspect <container>
docker run -it <image> /bin/sh
```

FIX

- Check logs for errors
- Verify environment variables
- Correct the command/entrypoint
- Test interactively

2 APPLICATION CANNOT CONNECT TO DATABASE

The application inside the container cannot reach the database.

COMMON CAUSES

- Wrong hostname or port
- Database not running
- Network misconfiguration
- Firewall or security group blocking

DIAGNOSTIC COMMANDS

```
docker ps
docker logs <app>
docker exec -it <app> sh
ping <db-host>
nc -zv <db-host> <port>
```

FIX

- Use correct database hostname
- Ensure database container is running
- Check Docker network
- Verify credentials
- Check firewall/security rules

3 CONTAINER PORT IS UNREACHABLE

You cannot access the application from your host or network.

COMMON CAUSES

- Port not published (-p)
- Wrong port mapping
- Application not listening on expected port
- Firewall blocking the port

DIAGNOSTIC COMMANDS

```
docker ps
docker port <container>
docker logs <container>
ss -tuln | grep <port>
```

FIX

- Check port mapping with -p
- Verify application is listening
- Check docker network and firewall
- Use correct host IP and port

4 DOCKER IMAGE FAILS TO BUILD

The docker build command fails with an error.

COMMON CAUSES

- Syntax error in Dockerfile
- Missing files in build context
- Network or dependency download issues
- Using the wrong base image or tag

DIAGNOSTIC COMMANDS

```
docker build .
docker build --no-cache .
docker build --progress=plain .
docker system df
```

FIX

- Fix Dockerfile syntax
- Ensure required files exist
- Check build context (.dockerignore)
- Use --no-cache and review error logs
- Verify base image and network access

5 CONTAINER RUNS OUT OF MEMORY

The container is killed or becomes unresponsive due to high memory usage.

COMMON CAUSES

- Memory limit too low
- Memory leak in application
- Application using more memory than expected

DIAGNOSTIC COMMANDS

```
docker stats
docker inspect <container>
docker logs <container>
dmesg | grep -i kill
```

FIX

- Increase memory limit (--memory)
- Optimize application memory usage
- Check for memory leaks
- Monitor resource usage regularly

6 PERMISSION DENIED INSIDE A CONTAINER

You get permission denied errors when accessing files or directories.

COMMON CAUSES

- Running as non-root user
- Wrong file or directory permissions
- Read-only filesystem
- Volume permission issues

DIAGNOSTIC COMMANDS

```
docker exec -it <container> whoami
docker exec -it <container> id
docker exec -it <container> ls -l <path>
docker inspect <container>
```

FIX

- Use correct user or set permissions
- Adjust file ownership (chown)
- Mount volumes with correct permissions
- Disable read-only if not required

7 DNS RESOLUTION FAILS

The container cannot resolve domain names.

COMMON CAUSES

- Incorrect DNS configuration
- No internet access
- Custom network without DNS
- DNS server unreachable

DIAGNOSTIC COMMANDS

```
docker exec -it <container> cat /etc/resolv.conf
docker exec -it <container> nslookup google.com
docker exec -it <container> ping google.com
docker network inspect <network>
```

FIX

- Use a working DNS server
- Check container network settings
- Verify internet access
- Specify --dns if needed

8 IMAGE PULL FAILS

The docker pull command fails to download the image.

COMMON CAUSES

- Incorrect image name or tag
- Private registry authentications required
- Rate limiting (e.g. Docker Hub)

DIAGNOSTIC COMMANDS

```
docker pull <image>:<tag>
docker login
docker logout
docker system df
```

FIX

- Check image name and tag
- Verify network and DNS
- Login to private registry
- Check registry access and quotas
- Use a different mirror if needed

DIAGNOSTIC COMMAND SELECTION AND VERIFICATION WORKFLOW

1 IDENTIFY THE SYMPTOM

- Understand the error.
- Check recent changes.
- Gather relevant logs.

2 COLLECT INFORMATION

- Use docker ps, logs, inspect.
- Check container status and events.
- Verify configuration and environment.

3 ANALYSE AND ISOLATE

- Identify the root cause.
- Test connectivity and resources.
- Reproduce the issue if possible.

4 APPLY THE FIX

- Make the necessary changes.
- Restart the container or rebuild.
- Validate the solution.

5 VERIFY AND MONITOR

- Confirm the issue is resolved.
- Monitor logs, resource usage.
- Put preventive measures in place.